

# Andy Issues

## End-to-End System Walkthrough

---

A backlog + sandbox management microservice in the Andy ecosystem.

*Designed for engineers who have never seen this service before.*

## What is Andy Issues?

---

A **microservice** that manages a hierarchical backlog (Repositories → Epics → Features → Stories) and dev-container **sandboxes** — with bidirectional sync to GitHub and Azure DevOps, LLM-assisted backlog generation, and event-driven integration across the Andy ecosystem.

**Three entry points into the same business logic:**

1. **REST API** (Swagger at `/swagger`) — used by the Angular SPA
2. **gRPC** — used by other services
3. **MCP** (Model Context Protocol) — used by AI agents

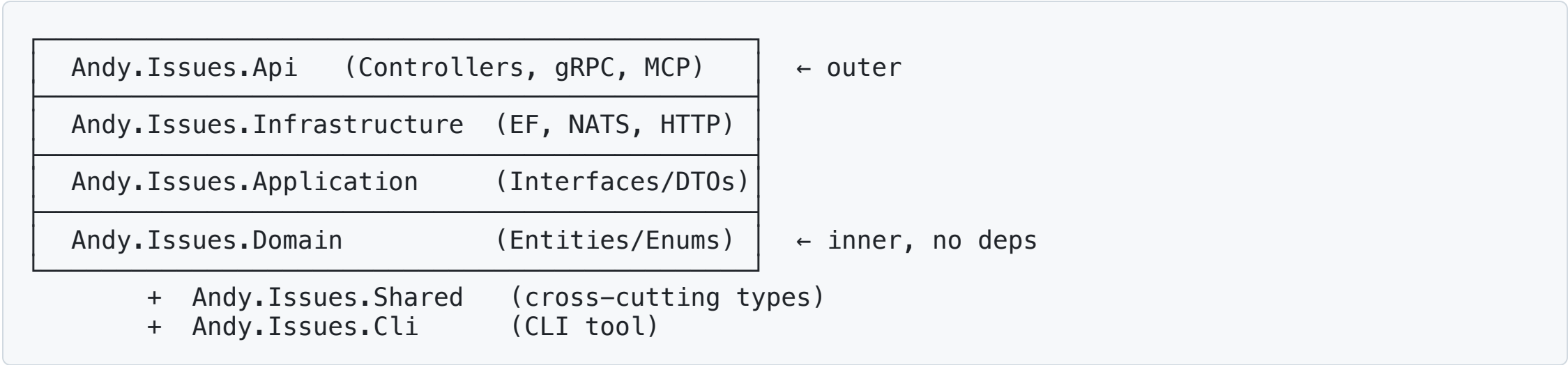
All three share one service layer. No duplication.

# Tech Stack at a Glance

---

| Layer         | Technology                                  |
|---------------|---------------------------------------------|
| Runtime       | .NET 8.0                                    |
| API           | ASP.NET Core (REST + gRPC + MCP)            |
| Frontend      | Angular 18 (standalone, OIDC)               |
| Database      | PostgreSQL (prod) / SQLite (embedded)       |
| ORM           | Entity Framework Core 8                     |
| Auth          | OAuth2/OIDC via Andy Auth (JWT Bearer)      |
| Authorization | Andy RBAC + JWT claims                      |
| Messaging     | NATS JetStream (prod) / InMemory (dev)      |
| Telemetry     | OpenTelemetry (OTLP)                        |
| Tests         | xUnit, WebApplicationFactory, Karma/Jasmine |

# Clean Architecture — Five Layers



**Dependency rule:** outer → inner only. Domain depends on *nothing*.

## Repository Layout

---

```
andy-issues/
├── src/
│   ├── Andy.Issues.Domain/      ← entities, enums
│   ├── Andy.Issues.Application/ ← interfaces, DTOs
│   ├── Andy.Issues.Infrastructure/ ← EF, NATS, clients
│   ├── Andy.Issues.Api/        ← REST/gRPC/MCP
│   └── Andy.Issues.Shared/
├── client/                      ← Angular 18 SPA
├── tools/Andy.Issues.Cli/       ← System.CommandLine
├── tests/                       ← Unit + Integration
├── config/                     ← auth + rbac seeds
├── docs/                       ← MkDocs + ADRs
└── docker-compose*.yaml
```

## The Domain — 4-Tier Hierarchy

---

```
Repository      (GitHub or AzureDevOps)
└─ Epic         (high-level theme)
   └─ Feature    (capability group)
      └─ UserStory (leaf; has Status + StoryPoints)
```

Plus adjacent aggregates:

- **Sandbox** — dev container per (Repository, Branch)
- **LinkedProvider** — stored GitHub/Azure tokens
- **OutboxEntry** — transactional outbox (ADR 0001)
- **LlmSetting, McpServerConfig, ArtifactFeedConfig**

## Domain Model — Key Details

---

**UserStory** ( `src/Andy.Issues.Domain/UserStory.cs` )

- `Status` enum: `Draft` → `Ready` → `InProgress` → `InReview` → `Done`
- `SetStatus()` **enforces** the state machine (no `Done`→`Draft`)
- `StoryPoints`, `AcceptanceCriteria`, `PullRequestUrl`
- `AzureDevOpsWorkItemId` for bidirectional sync

**Repository** owns `CodeIndexStatus` (`NotIndexed` / `Indexing` / `Indexed`) — used by the LLM-driven draft backlog generator.

**OutboxEntry** fields: `Subject`, `PayloadJson`, `CorrelationId`, `CausationId`, `Generation`, `PublishedAt`, `AttemptCount`, `LastError`.

## Application Layer — Service Contracts

---

Key interfaces (`src/Andy.Issues.Application`):

- `IBacklogService` — Epic/Feature/Story CRUD + status transitions
- `IRepositoryService` — list/create/sync GitHub + Azure DevOps
- `ISandboxService` — create/delete (proxies andy-containers)
- `IBacklogGitHubImportService` — import issues → backlog
- `IBacklogAzureDevOpsSyncService` — push local backlog → AzDO
- `IDraftBacklogGenerator` — LLM + code index → proposed backlog
- `IMessageBus` — publish/subscribe abstraction
- `IRepositoryAccessGuard` — authorization check
- `IUserDirectory`, `IPermissionChecker`, `ISecretStore`

**DTOs are `record` types** — immutable, value-based equality.

## Infrastructure — Database

**AppDbContext** — 11 DbSet, dual-provider support.

| Provider   | When                               |
|------------|------------------------------------|
| PostgreSQL | default / production (port 5443)   |
| SQLite     | embedded mode (Conductor bundling) |

- `DateTimeOffset` → binary for SQLite to preserve ordering
- Auto-migration runs in Development mode
- Design-time factory: `Infrastructure/Data/DesignTimeDbContextFactory.cs`

**Migrations** (5 to date):

`InitialDomain` → `UserDirectoryEntry` → `AzureDevOpsWorkItemId` → `SandboxContainerId→String` → `OutboxEntry`

## Infrastructure — Messaging (the heart of cross-service)

---

Two `IMessageBus` implementations selected by `Messaging:Provider`:

- `InMemoryMessageBus` — in-process queue, dev/test
- `NatsMessageBus` — NATS JetStream, production

Headers we propagate:

- `Nats-Msg-Id` — outbox row id (dedup key)
- `Andy-Correlation-Id` — traces the whole causal chain
- `Andy-Causation-Id` — the event that caused this one
- `Andy-Generation` — hop count, prevents infinite loops

Malformed messages go to a **DLQ**. Manual ACK for explicit semantics.

## The Transactional Outbox (ADR 0001)

---

**Problem:** If we write to the DB *and* publish to NATS separately, a crash between them leaves state divergent.

**Solution:** Write event row + domain change in the same DB transaction, then let a background dispatcher publish.

```
BacklogService.AddStoryAsync(...)
    db.UserStories.Add(story);
    StoryEventOutbox.AppendStoryEvent(story, ..., Created);
    await db.SaveChangesAsync(); ← atomic!
```

Background: **OutboxDispatcher** polls `Outbox WHERE PublishedAt IS NULL`, batch-publishes with exponential backoff retry.

**Result:** at-least-once delivery + idempotent consumers = effectively-once.

## Event Subject Taxonomy

---

```
andy.issues.events.<entity>.<id>.<kind>
```

Concrete examples:

- `andy.issues.events.story.{storyId}.created`
- `andy.issues.events.story.{storyId}.readied`
- `andy.issues.events.story.{storyId}.done`
- `andy.issues.events.repository.{repoId}.created`
- `andy.issues.events.sandbox.{sandboxId}.created`

**Consumed** from andy-containers:

- `andy.containers.events.run.>` → `ContainerRunEventConsumer` updates story status on terminal events (finished/failed/cancelled), with a 1024-slot dedup ring buffer.

Payloads are JSON, **snake\_case** (see `EventJson.Options`).

# API Layer — REST Controllers

8 controllers under `/api`:

| Controller                | Responsibility                     |
|---------------------------|------------------------------------|
| BacklogController         | Epic/Feature/Story CRUD + status   |
| RepositoriesController    | Repo list/sync/generate-backlog/PR |
| SandboxesController       | Sandbox lifecycle                  |
| LinkedProvidersController | GitHub/Azure token mgmt            |
| McpController             | MCP server config                  |
| ArtifactController        | Artifact feed config               |
| UsersController           | User directory / suggestions       |
| HelpController            | Markdown help topics               |

All `[Authorize]` by default; `[AllowAnonymous]` only for `/health`.

## API Layer — gRPC & MCP

---

**gRPC (3 services)** — mirrors REST for service-to-service:

- `BacklogGrpcService` (`/Protos/backlog.proto`)
- `RepositoriesGrpcService`
- `SandboxesGrpcService`

Proto uses `google.protobuf.wrappers` for optional fields.

**MCP (`ServiceTools`)** — 30+ tools for AI agents:

- `ListRepositories`, `GetBacklog`, `CreateStory`, `UpdateStoryStatus`
- `SyncGitHubRepositories`, `SyncAzureDevOpsRepositories`
- `CreateSandbox`, `CreatePullRequest`
- Exposed at `/mcp` over HTTP transport

All paths funnel into the same `IBacklogService` / `IRepositoryService` / etc.

## Program.cs — Startup Composition

---

```
builder.Services
    .AddAppDatabase(cfg)           // Postgres or SQLite
    .AddIssuesMessaging(cfg)       // InMemory or NATS
    .AddAndyAuth(cfg)             // JWT Bearer validation
    .AddAndyRbac(cfg)             // optional HTTP client
    .AddAndySettings(cfg)         // centralized config
    .AddOpenTelemetry(...)        // traces + metrics
    .AddSignalR()                 // /hub/board realtime
    .AddGrpc()
    .AddMcpServer().WithHttpTransport().WithToolsFromAssembly();

app.MapControllers();
app.MapGrpcService<BacklogGrpcService>();
app.MapMcp("/mcp");
app.MapHub<BoardHub>("/hub/board");
app.MapFallbackToFile("index.html"); // Angular SPA
```

## Authentication & Authorization

---

- **Andy Auth** (`https://localhost:5001`) issues JWTs via OAuth2/OIDC.
- API validates Bearer tokens; extracts `sub` claim via `User.RequireUserId()`.
- **Dev bypass**: when `AndyAuth:Authority` is empty, all endpoints open — for local dev only.
- **Test user** (seeded in non-prod): `test@andy.local` / `Test123!`
- **Andy RBAC** (optional, `https://localhost:5003`): app code `andy-issues` with admin/user/viewer roles.
- **ClaimsPermissionChecker** falls back to JWT claims if RBAC isn't reachable.
- **BearerForwardingHandler** propagates the caller's JWT to downstream HTTP calls (andy-containers, etc.).

## External Dependencies

---

| Service               | Port     | Role                          |
|-----------------------|----------|-------------------------------|
| Andy Auth             | 5001     | OAuth2/OIDC identity provider |
| Andy RBAC             | 5003     | Role-based access control     |
| Andy Settings         | 5300     | Centralized configuration     |
| Andy Containers       | via HTTP | Sandbox container runtime     |
| Andy Code Index       | via HTTP | Code indexing for LLM prompts |
| NATS                  | 4222     | Event streaming (JetStream)   |
| PostgreSQL            | 5443     | Primary data store            |
| GitHub / Azure DevOps | public   | Backlog sync targets          |

Every external client has a **local fallback** (e.g. `LocalSettingsClient`, `NullCodeIndexClient`).

## Data Flow — Creating a Story (1/2)

---

1. **Client** → `POST /api/features/{featureId}/stories`
2. **BacklogController.CreateStory** → `IBacklogService.AddStoryAsync(...)`
3. **BacklogService**:
  - Fetches Feature + Epic (FK chain to Repository)
  - `IRepositoryAccessGuard.CanViewAsync(repositoryId, userId)` — 401 if not
  - Creates `UserStory { Id, Status=Draft, CreatedAt=UtcNow }`
  - `StoryEventOutbox.AppendStoryEvent(story, ..., Created)` — queues event row
  - `await db.SaveChangesAsync()` — **atomic** commit of both
  - `_notifier.EpicAddedAsync()` — SignalR push to connected browsers
  - Returns `UserStoryDto`

HTTP 201 Created returns to client.

## Data Flow — Creating a Story (2/2)

---

### 4. **OutboxDispatcher** (background, periodic):

- `SELECT * FROM Outbox WHERE PublishedAt IS NULL ORDER BY CreatedAt LIMIT batchSize`
- For each row → `IMessageBus.PublishAsync(subject, payload, headers)`
- On success → `PublishedAt =.UtcNow`. On failure → backoff + retry.

### 5. **NATS JetStream** persists message on subject

`andy.issues.events.story.{storyId}.created`.

### 6. **Subscribers** (elsewhere in the ecosystem) consume at-least-once with dedup via `Nats-Msg-Id` = outbox row id.

**Guarantees:** no dual-write race, correlation chain preserved, idempotent reprocessing.

## The Angular Frontend

---

`client/src/app/` — standalone components, OIDC via `angular-auth-oidc-client`.

**Routes** (lazy-loaded):

- `/dashboard` — landing (requires auth)
- `/repositories` — repo list + sync actions
- `/backlog/:repoId` — tree view of Epics/Features/Stories
- `/sandboxes` — dev container management
- `/settings`, `/help`, `/callback`

**HTTP interceptor** (`auth.interceptor.ts`): attaches Bearer token from `OidcSecurityService.getAccessToken()` to every `/api/*` request.

**SignalR** `/hub/board`: real-time board updates when others edit the backlog.

## CLI Tool ( `Andy.Issues.Cli` )

---

Built with `System.CommandLine`. Authenticates via `--token` flag.

```
dotnet run --project tools/Andy.Issues.Cli -- \
  repos list --api-url https://localhost:5410

dotnet run --project tools/Andy.Issues.Cli -- \
  backlog add-story --feature-id <guid> --title "... "

dotnet run --project tools/Andy.Issues.Cli -- \
  sandboxes create --repository-id <guid> --branch main
```

Command groups: `repos` · `backlog` · `sandboxes` · `mcp` · `artifact-feeds`.

Each group mirrors the REST surface; the CLI is a thin shell over `ApiClient`.

## Testing Strategy

---

**Unit tests** ( `tests/Andy.Issues.Tests.Unit` ) — EF Core InMemory:

- `BacklogMappingTests`, `SecretMaskingTests`
- `OutboxDispatcherTests` — batch, retry, backoff
- `MessageHeadersTests` — correlation/causation/generation
- `ContainerRunEventConsumerTests` — dedup ring buffer
- `ServiceToolsTests`, `HelpToolsTests` — MCP
- `CommandParsingTests` — CLI

**Integration tests** ( `tests/Andy.Issues.Tests.Integration` ):

- `WebApplicationFactory<Program>` + `TestAuthHandler`
- In-memory SQLite DB per test
- NATS integration job in CI (Story 15.8)

**Policy:** `TreatWarningsAsErrors=true` · run `dotnet test` before claiming done.

## Configuration Snapshot

---

`appsettings.json` (key sections):

```
{
  "Database": { "Provider": "PostgreSql" },
  "AndyAuth": { "Authority": "https://localhost:5001",
                "Audience": "urn:andy-issues-api" },
  "Rbac":      { "ApiBaseUrl": "https://localhost:5003",
                "ApplicationCode": "andy-issues" },
  "AndySettings": { "ApiBaseUrl": "https://localhost:5300" },
  "Messaging": { "Provider": "InMemory",
                "Nats": { "Url": "nats://localhost:4222",
                        "StreamName": "andy-issues" } },
  "OpenTelemetry": { "ServiceName": "andy-issues-api" }
}
```

Override per env via `appsettings.{Environment}.json` or env vars.

## Deployment — Docker

---

### Multi-stage Dockerfile:

1. `node:22-alpine` → `npm run build` → Angular SPA in `dist/client/browser`
2. `dotnet/sdk:8.0` → `dotnet publish` → `/app/publish`
3. `dotnet/aspnet:8.0` → runs as non-root `appuser`, ports 8080/8443

Supports corporate CA injection via `certs/` volume.

**`docker-compose.yml`**: postgres + andy-issues (port 5410).

**`docker-compose.embedded.yml`**: SQLite mode for Conductor.

```
docker compose up -d                # full stack
docker compose -f docker-compose.embedded.yml up # embedded
```

## Ports Cheat Sheet

---

| Port | Service                 |
|------|-------------------------|
| 5410 | Andy Issues API (HTTPS) |
| 5411 | Andy Issues API (HTTP)  |
| 4200 | Angular dev server      |
| 4203 | Angular in Docker       |
| 5443 | PostgreSQL              |
| 5001 | Andy Auth               |
| 5003 | Andy RBAC               |
| 5300 | Andy Settings           |
| 4222 | NATS                    |

## Observability & Operations

---

- **OpenTelemetry** instrumentation: ASP.NET Core + HttpClient + EF Core.
- OTLP export configurable via `OpenTelemetry:OtlpEndpoint`.
- **Health check**: `GET /health` (AllowAnonymous) returns `{ status, timestamp }`.
- **Swagger UI**: `/swagger` with Bearer security scheme.
- **SignalR hub**: `/hub/board` (for realtime UI updates).
- **MCP endpoint**: `/mcp` with permissive CORS for AI agents.

**Secret scanning**: pre-commit hook + Gitleaks in CI + GitHub native scanning. Dev defaults ( `_dev_password`, `Test123!` ) allowlisted in `.gitleaks.toml`.

## Recent Work — Story 15.x (NATS Rollout)

---

Recent commits (main):

- **15.5** — Publish `andy.issues.events.sandbox.*`
- **15.6** — Subscribe to `andy.containers.events.run.*`
- **15.7** — NATS provider wire-up (Infrastructure)
- **15.8** — NATS integration tests + CI job

This hardened the outbox + NATS path end-to-end. The InMemory bus remains the default for local dev and unit tests.

**ADR 0001** ( `docs/adr/` ) documents the transactional-outbox decision.

## Mental Model — One Sentence Each

---

- **Domain:** a backlog tree + sandboxes per repo, with forced status transitions.
- **Application:** interfaces the API and tests code against — no EF, no HTTP.
- **Infrastructure:** how those interfaces actually reach Postgres, NATS, GitHub.
- **API:** three faces (REST / gRPC / MCP) over one service layer.
- **Messaging:** write event rows transactionally, let a dispatcher publish them.
- **Auth:** JWT from Andy Auth, forwarded to downstream services verbatim.
- **Frontend:** thin Angular SPA; all state lives in the API.
- **CLI:** scriptable surface that hits the same REST API.

## Where to start reading

1. `src/Andy.Issues.Domain/UserStory.cs` — understand the core invariant
2. `src/Andy.Issues.Application/IBacklogService.cs` — the service contract
3. `src/Andy.Issues.Infrastructure/Services/BacklogService.cs` — the real impl
4. `src/Andy.Issues.Api/Controllers/BacklogController.cs` — the HTTP surface
5. `src/Andy.Issues.Infrastructure/Messaging/OutboxDispatcher.cs` — the publishing loop
6. `src/Andy.Issues.Api/Program.cs` — how it all wires together

**Questions?** `docs/` has MkDocs pages + ADRs. Swagger at `/swagger` is authoritative for the REST surface.